MEMORANDUM

To: Charlie Refvem, Lecturer, Department of Mechanical Engineering, Cal Poly SLO
crefvem@calpoly.edu

From: Quinn Patterson
qpatters@calpoly.edu
Jake Rowen
jhrowen@calpoly.edu

Team: MECHA23

Date: 1/13/26

RE: **Lab 0x00 Findings**

Table of Contents

1. Abstract
2. Figures
3. Code Attachment

Abstract

In this lab we familiarized ourselves with the NUCLEO-L476RG board and its basic functionalities. We learned how to flash firmware and upload micropython code through the Shoe of Brian. A simple RC circuit was then attached and the board was utilized to collect data during the circuits step response. From our data we determined our theoretical time constant to be 0.094s, and the measured constant to be 0.085s. This error comes out to about 12% and is close enough to be acceptable for our purposes.

Figures

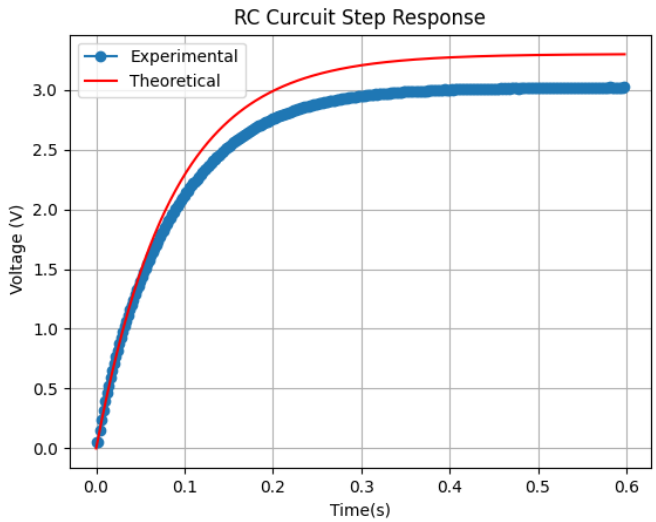

Figure 1. Theoretical and Experimental Step Responses

Figure 1 showcases our step response compared to the theoretical max. Since the capacitor is not ideal it can never truly reach its complete maximum, which is visualized in the plot.

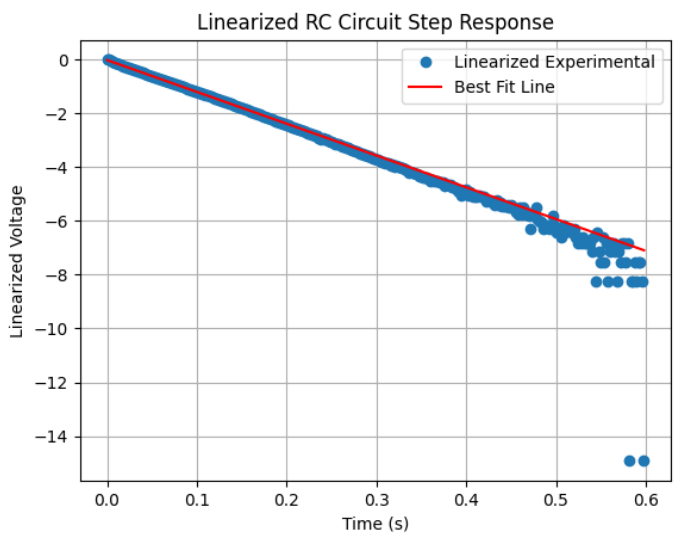

Figure 2. Linearized Step Response with Line of Best Fit

Figure 2 showcases our linearized experimental data and a line of best fit we use to calculate the time constant. The line of best fit was calculated off the first 20% of the data because the spread after that begins to widen and can throw off our data.

Code Attachments

1. Measure and record step response

```
import pyb
from pyb import Pin
from pyb import ADC
from pyb import Timer
from array import array
from time import sleep
PC0 = Pin(Pin.cpu.C0, Pin.IN)
PC1 = Pin(Pin.cpu.C1, Pin.OUT)

adc = ADC(PC0)

tim = Timer(6, freq = 500)

data = array('H', [0] * 300)
idx = 0
done = False
PC1.low()

def tim_cb(tim):
    global data, idx, done
    val = adc.read()
    if idx == 1:
        PC1.high()
    if idx < len(data):
        data[idx] = val
        idx += 1
    else:
        tim.callback(None)
        done = True

# Toggle input and measure step response with ADC
tim.callback(tim_cb)

while not done:
    pass

for idx, val in enumerate(data):
    print(f"{idx/500}, {val*(3.3/4095)}")

PC1.low()
```

2. Plot data from CSV file

```
from matplotlib import pyplot as plt
import math
import numpy as np

from numpy import array

with open('lab1_data.csv', 'r') as data:
    items = data.readlines()
    headers = items[0].strip().split(',')

data = []
for i in range(1, len(items)):
    values = items[i].strip().split(',')
    subvalues = []
    for val in values:
        if '#' in val:
            subvals = val.partition('#')
            for subval in subvals:
                if subval.strip() != '':
                    subvalues.append(subval.strip())
        else:
            subvalues.append(val)
    data.append(subvalues)

clean_data = []
for i in range(len(data)):
    row = []
    for j in range(len(data[i])):
        try:
            row.append(float(data[i][j]))
        except ValueError:
            if len(data[i])<= 2 and (len(data[i]) == 0 or
data[i][j] == ''):
                print(f"Empty line found on line {i+2}, skipping
line.")
                continue
            elif "#" in data[i][j]:
                print(f"Comment found on line {i+2}")
                continue
            else:
                print(f"Invalid data on line {i+2}, skipping
line.")
                continue
    if len(row) >= len(headers):
        clean_data.append(row)
```

```
time = [row[0] for row in clean_data]
height = [row[1] for row in clean_data]

# time constant

step_vlt = .632*height[-1]

tau = 0.08461548312758245

theoretical = [0]*len(time)

for i in range(len(time)):
    theoretical[i] = 3.3*(1 - math.exp(-1*time[i]/tau))

plt.plot(time, height, label= 'Experimental',marker='o')
plt.title('RC Curcuit Step Response')
plt.xlabel(headers[0])
plt.ylabel(headers[1])
plt.grid()
plt.plot(time, theoretical, label='Theoretical', color='red')
plt.legend()
plt.show()

# linearized step response

linearized = [0]*len(height)
for i in range(len(height)):
    linearized[i] = math.log(abs(1-
(height[i]/(max(height)+.000001))))

# best fit data
coefficients = np.polyfit(time[0:60], linearized[0:60], 1)

best_fit = [coefficients[0]*t + coefficients[1] for t in time]

print("Time constant from best fit: ", -1/coefficients[0])

plt.plot(time, linearized, label='Linearized Experimental',
linestyle='None', marker='o')
plt.title('Linearized RC Circuit Step Response')
plt.xlabel('Time (s)')
plt.ylabel('Linearized Voltage')
plt.plot(time, best_fit, label='Best Fit Line', color='red')
plt.grid()
plt.legend()
plt.show()
```